

Defense guide: chunk-recall-audit

For reading before an interview. Every number here is re-measured by `tools/collect_metrics.py` and checked against this file by `tools/check_numbers.py`, so a stale sentence fails the build rather than getting read aloud.

The 30 second version

"Retrieval metrics are computed on the chunks the chunker produced, so they cannot see the answers the chunker destroyed. A chunk holding half an answer counts as a hit. I built a corpus with exact answer spans and measured both quantities on the same rankings. On 200 character chunks the standard metric reports a recall of 1.0 and only 0.2917 of questions have any single retrieved chunk that holds the whole answer."

Then stop.

The question that usually comes next is "so what do I do about it", and the answer is the interesting part: for the configurations most pipelines are running, nothing you do to the retriever helps.

The claims, and how each one is proved

Claim 1: the two metrics are far apart

Command: `python -m chunkaudit audit --size 200 --stride 200`

The worst is fixed-200-150: the standard metric reports a recall of 1.0, the pipeline actually answers 0.2528, and 269 of them cannot answer on their own out of 360 questions. That is a gap of 0.7472.

If pushed on "maybe your k is too small": the gap is 0.85 at k equal to one and still 0.7472 at twenty. Span complete recall goes flat from k equal to 3 while the standard metric climbs to one. Retrieving more copies of a broken answer does not assemble one.

Claim 2: almost nobody's retrieval problem is a retrieval problem

Command: `python experiments/exp02_who_can_fix_this.py`

Across 13 chunkings with BM25 there were 1663 failures. 1661 were the chunker's and 2 were the retriever's. 10 of the thirteen configurations have no retriever fixable failure at all.

If pushed on "that is because BM25 is good on your corpus": the weaker hashed vector retriever produced 177 retrieval failures against the same 1661 from chunking. Making the retriever materially worse does not change which cause dominates.

Claim 3: the overlap is a guarantee threshold, and it is exact

Command: `python experiments/exp03_the_guarantee_threshold.py`

For uniform character windows of size S and stride T, an answer of length L survives at every position if and only if L is at most S minus T plus one. At size four hundred and stride three hundred that is 101 characters at size four hundred, verified across all 3899 positions, with one character longer already failing somewhere.

Above it, survival is a lottery whose odds are computable. At 150 characters a chunking with no overlap keeps 0.652 of positions.

If pushed on "so just add overlap": there is 1 case in this plan where adding overlap lowered the ceiling. Going from fixed-200-200 to fixed-200-150 the ceiling falls by 0.0389 while the index grows by 293 more chunks.

Window starts are multiples of the stride, so a smaller stride is not a superset of a larger one.

Claim 4: the ceiling bounds every retriever, and it is tight

Command: `python experiments/exp05_which_lever_matters.py`

A retriever cannot return a chunk that was never made, so the ceiling bounds BM25, a dense encoder, a reranker, and anything not yet written. It is also attained: an oracle that ranks by overlap with the answer reaches it on every chunking, with a worst absolute difference of 0.0.

If pushed on "you never tested a real embedding model": correct, and it is in ADR-003 and in the README's own section. The model weights host was unreachable from the build environment. What that does not threaten is this claim, because the bound holds for any retriever and the oracle shows it is reached. What it does cost is named: the retriever range in claim 6 covers two lexical methods, so the ratio of 4 times as far is an upper estimate of the chunker's relative importance rather than a settled figure.

Claim 5: the chunker is the bigger lever

Changing between the two real retrievers moves span complete recall by at most 0.1972. Changing the chunker moves it by up to 0.7917.

If pushed on "so retrieval does not matter": it does. The gap between BM25 and a seeded shuffle reaches 0.9722, so the retriever is not irrelevant. It is the smaller lever, and the point is where to spend next rather than which stage to ignore.

Questions that are meant to be hard

Is this just recall at k with extra steps? No, and the difference is stateable in one sentence: recall at k asks whether any retrieved chunk touches the answer, and this asks whether any single one holds it. What is mine beyond that: the containment ceiling and the proof it is attained, the three way failure attribution, the closed form for the guarantee threshold checked by exhaustion, the non monotonicity result with a constructed counterexample, and the receipts pipeline that fails the build when a document quotes a number the code no longer produces. Around a thousand statements of source, 232 tests, 99.2 percent line coverage.

Your corpus is generated. Does any of this transfer? Two kinds of claim. The ceiling arithmetic and the guarantee threshold are properties of the chunker and transfer to any corpus. The magnitudes do not: 0.2917 at 200 characters is a property of these answer lengths. What a real corpus would change is the answer length distribution, and the whole point of the guarantee threshold is that a team can compute their own number from theirs.

Why is there no transformer in a repository about chunking for RAG? Because the model weights host was unreachable from the environment I built it in, so I wrote the path, fenced it behind an extra, tested it with the module made unimportable, and made the tool itself print that it has never been run. The reason I was willing to ship without it is that the ceiling bounds every retriever including that one, and the oracle shows the bound is attained.

What is the weakest part? The corpus is generated, so every magnitude is a property of a corpus I wrote. Second weakest, and the more interesting answer: the retriever range in claim 6 covers two lexical methods, so the claim that the chunker is 4 times the lever is an upper estimate rather than a settled ratio.

What would you do differently with more time? Run it on a real question answering set with labelled spans, because that turns every magnitude from illustrative into real. Then turn the guarantee threshold into advice rather than a table: given a measured answer length distribution, the right chunk size and overlap are an arithmetic

problem with an exact answer. Then price the downstream cost of longer chunks, which is the column claim 5 is missing.

Did anything go wrong while you built it? Four things, and the useful one is the ceiling violation. A measured span complete recall of 0.850 came in against a ceiling of 0.828, which cannot happen, because the ceiling is an upper bound by construction. The cause was comparing character offsets across documents: offsets are per document, so a chunk covering characters 0 to 800 of one document numerically contains an answer at characters 100 to 500 of another. The invariant is now a test and it is the most valuable one in the repository, because it can only fail when a measurement is impossible.

Why should I trust the whole document row, which wins on your own metric? You should not treat it as the recommendation, and the repository says so. It has a perfect ceiling and it ships 7662 characters per question, against the best real alternative that reaches 0.8278 on 3731 characters. What those extra characters cost in generation quality, latency and token spend is not measured here, and that is in the section on what the repository cannot establish.

Things to say, and things not to say

Say:

- "the ceiling bounds every retriever, and the oracle shows it is attained."
- "1661 of 1663 failures were the chunker's, on this corpus, at k equal to 5."
- "the overlap is a guarantee threshold on answer length, and here is the closed form."
- "the chunker is the bigger lever, and that ratio is an upper estimate because I only ran lexical retrievers."

Do not say:

- "zero retrieval failures." Say ten of thirteen chunkings had none over 360 questions, which supports below 0.28 percent and nothing stronger.
- "chunking is more important than retrieval." Say it is the bigger lever on this corpus, and that the gap to a seeded shuffle reaches 0.9722, so retrieval is not irrelevant.
- "my hashed retriever is a dense encoder." It hashes character n grams with inverse document frequency weighting. Nothing in the repository calls it neural.
- "99.2 percent coverage means it is correct." It means the lines ran. The ceiling invariant is the test that means something.

The live demo, five commands

```
# 1. The plan, and which retriever has never been run here.
python -m chunkaudit plan

# 2. A common chunk size, and what it destroyed. Exit 1.
python -m chunkaudit audit --size 200 --stride 200

# 3. The same corpus with whole document chunks. Exit 0, and see the cost.
python -m chunkaudit audit --strategy document

# 4. Every chunking, sorted worst ceiling first.
python -m chunkaudit sweep

# 5. Every published figure re-measured and checked against the documents.
make receipts
```